> 本篇是docker的相关笔记，本质是速记，把一些命令记下来，概念就没有了

# Docker命令

> docker的核心两大概念就是**镜像(image)和容器(container)**，所以就分这两点速记一下

## Docker镜像命令

1. **查看本地主机上所有镜像**

   ```
   docker images
   ```

   参数：

   -a：显示所有镜像

   -q：只显示镜像的id

2. **搜索镜像**

   ```
   docker search
   ```

   参数：

   --filter筛选

3. **拉取镜像**

   ```
   docker pull 镜像[:tag]
   ```

4. **删除镜像**

   ```
   docker rmi
   ```

## Docker容器

1. **新建容器并启动**

   ```
   docker run [参数] image
   ```

   --name='名字'　　容器名字

   -d　**后台运行**

   -it　交互方式运行，进入查看内容

   -p　指定端口，格式→外网端口:容器端口

   -P　随机指定端口

   -e　**设置环境变量**，比如**-e MYSQL_ROOT_PASSWORD=123456**

2. **退出**

exit停止并退出

Ctrl+P+Q不停止退出

3. **列出所有正在运行的容器**

`docker ps`

-a 把曾经运行过的也一同打印

-n=数字  显示最近几次创建的容器

-q  只显示编号

4. **删除容器**

```
docker rm 容器
docker rm -f $(docker ps -aq)
```

-aq表示把全部编号获取

没法删在运行的容器

5. **启动相关**

```
docker start
docker restart
docker stop
docker kill(强制停止)
```

## 常用命令

1. **查看日志**

```
docker logs [-f -t] 容器
```

2. **查看容器内数据**

```
docker inspect
```

3. **进入当前正在运行的容器**

```
docker exec -it 容器id 命令行路径#(开一个新终端)

docker attach 容器id#(直接用旧终端)
```

4. **从容器内复制文件到主机**

```
docker cp 容器编号:目录 主机目录
```

5. **配置数据卷**

```
docker run -it -v 宿主机目录:容器内目录
```

6. **配置数据卷容器**

```
docker run -it --name=名字 -v 容器内目录
```

**把容器挂载到数据卷容器上**

```
docker run -it  --name=名字 --volumes-from 数据卷容器名字 镜像 命令行
```

# Dockerfile

> 以下没啥用（是不用dockerfile的情况如何修改镜像）

1. **把容器转为镜像**

```
docker commit 容器id 镜像名称:版本号
```

2. **把镜像转成压缩文件**

```
docker save -o 压缩文件名 镜像名称:版本号
```

3. **把压缩文件再转回镜像**

```
docker load -i 压缩文件名称
```

> 有用的来了

**dockerfile的作用就是描述如何构建一个镜像**

| 关键字 | 作用 | 说明 |
| --- | --- | --- |
| FROM | 指定父镜像 | 规定dockerfile基于哪一个镜像来构建 |
| MAINTAINER | 作者 | dockerfile的作者 |
| LABEL | 标签 | 一些image的基本信息 |
| RUN | 执行的命令 | 去执行一段命令 |
| CMD | 容器启动的命令 | 启动容器时默认命令<br>比如/bin/bash<br>这样就可以在run的时候省略这一段了 |
| ENTRYPOINT | 入口 | 容器启动后，第一个要执行的主程序 |
| COPY | 复制文件 | 创建时复制文件到镜像里 |
| ADD | 添加文件 | 创建的时候添加文件到镜像，可以是远程的 |
| ENV | 环境变量 | 指定创建时环境变量 |
| ARG | 构建参数 | |

| 关键字 | 作用 | 说明 |
| --- | --- | --- |
| EXPOSE | 暴露端口 | 定义容器运行的时候监听的端口 |
| WORKDIR | 工作目录 | 定义默认工作目录 |

### 例子

```
FROM openjdk:17
COPY target/app.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

### 用dockerfile创建镜像

```
docker build -f dockerfile文件路径 -t 镜像名称:版本 .
```

## docker compose

**Docker Compose 是一个用于定义和管理多个 Docker 容器的工具**（当然一个也行）

如果需要运行多个服务，就要创建多个容器，一个一个run，这很麻烦

要使用dockercompose，需要写一个配置文件**compose.yaml**（名字不能乱改，只能在官方认可的标准之间改）

```
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: example
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

| 主要配置 | 含义 |
| --- | --- |
| services | 定义项目中有哪些服务 |
| mysql | 服务的名字 |
| image | 指定使用哪个镜像 |
| environment | 设置容器运行所需的环境变量 |
| ports | 端口映射 |
| volumes | 数据挂载 |

然后就可以启动运行容器了（**使用这些命令要在那个配置文件文件夹里**）

**后台运行**

```
docker compose up -d
```

**停止但不删除**

```
docker compose stop
```

**查看容器状态**

```
docker compose ps
```